Week 5: Nested Loops and Arrays

ID1063 • Introduction to Programming in C

Instructor: Rakesh Venkat

August 2026

Recap from Week 4

- A while loop checks its condition before each iteration.
- A do-while loop executes its body at least once.
- break and continue change the flow of the nearest loop.
- A loop needs an initial value, a condition, and an update.

```
do {
    // Code to be executed
    // Update expression (e.g., i++)
} while (condition);
```

```
while (condition)
{
    // block of code to be executed
}
```

Recap: Example code

```
int i = 0;
while (i < 10) {
  if (i == 4) {
       continue;
  }
  printf("%d\n", i);
  i++;
}
```

- What does the above code do?

This week

- Nested while and do-while loops
- Arrays: storing several values of the same type
- Reading and processing an array using while loops

Double summation

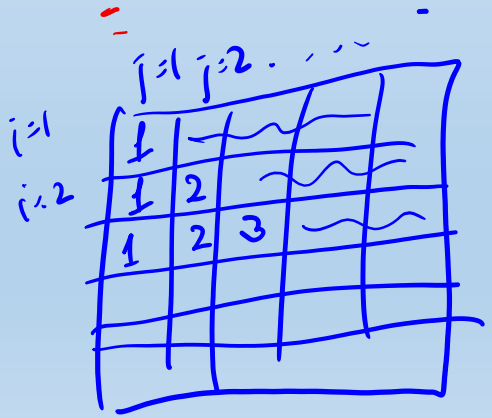
- What is the answer to the following:

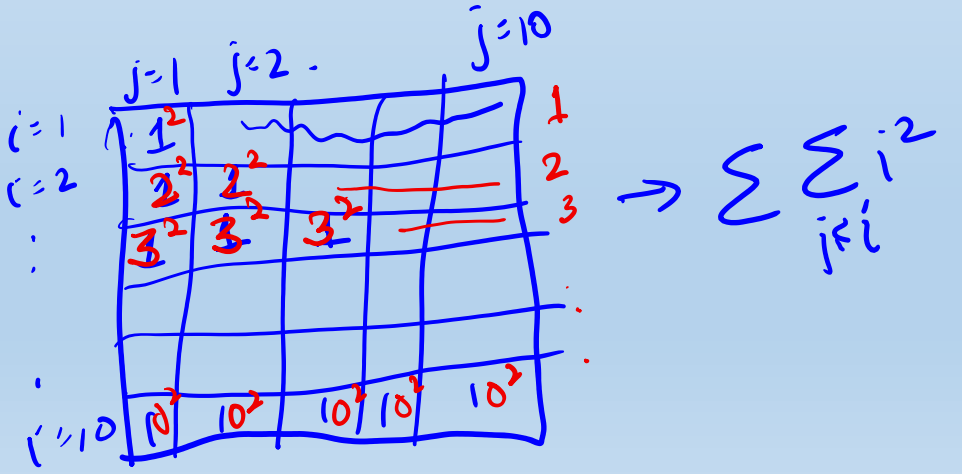
$$\sum_{1 \le i \le 10} \left(\sum_{1 \le j \le i} 1 \right)$$

$$= \sum_{i=1}^{10} i$$

$$\sum_{i=1}^{10} 1 = 10$$

$$\sum_{i=1}^{10} i = \text{sum of } 1 \ldots 10$$





$$\rightarrow \sum_{j \le i} i^2$$

Nested loops

- Nested : One loop *within* another
- Complete one inner task
- Repeat inner task

```c
#include <stdio.h>

int main() {
    // Outer while loop
    while (condition1) {
        // Code to be executed as long as condition1 is true

        // Inner while loop
        while (condition2) {
            // Code to be executed as long as condition2 is true
        }

        // Rest of the code within the outer while loop
    }

    return 0;
}
```

One loop inside another

- The outer loop chooses the current row.
- The inner loop completes all the work for that one outer iteration.
- After the inner loop finishes, the outer loop moves to the next iteration.
- The inner counter usually starts again for every new outer iteration.

A rectangle of stars

```
rows = 3, columns = 5


* * * * *
* * * * *
* * * * *
```

- Use while loop to print the above pattern on the screen

- Example code..

Caution: inner loops should reset its counter

```
int numrows=3, numcols=5;
int i = 1;
int j = 1;

while (i <= numrows)
{
    while (j <= numcolumns)
    {
        printf("* ");
        j = j + 1;
    }
    printf("\n");
    i = i + 1;
}
// Q: What is the output of the above code? What is the problem?
Ans: The variable j is not reset for each value of i. So the program prints…. (try out yourself and check).
```

One corrected version (only relevant lines shown)

```
int numrows, numcols;
numrows = 3;
numcols = 5;
int i=0,j=0;

while(i<numrows)
{
          j=0;  // reset the column counter j
         while(j<numcols) {
                   printf("* ");
                   j=j+1; //increment j
         }

         printf("\n"); // go to next line
         i=i+1; //increment i
}
```

Alternative:

```
int numrows, numcols;
              numrows = 3;
              numcols = 5;
              int i=0;

              while(i<numrows)
              {
                            int j=0;  // declare the column counter j
                            while(j<numcols) {
                                          printf("* ");
                                          j=j+1; //increment j
                            }

                            printf("\n"); // go to next line
                            i=i+1; //increment i
              }
```

Q: What is the difference between this and the previous code?

Ans: The variable j (column counter) is re-declared every time a loop iteration for I occurs. The instances of j are killed/unallocated after every iteration of i.

If you try to access j outside the while loop of i, the program will not compile. Try adding printf("%d",j); at the end of the above code and try to compile it.

Nested do-while loops

```
int row = 1;

do
{
    int column = 1;

    do
    {
        printf("* ");
        column = column + 1;
    }
    while (column <= numcolumns);

    printf("\n");
    row = row + 1;
}
while (row <= numrows);
```

Do While is not the perfect choice, unless at least one iteration is surely to be executed.

Example 2: Number triangle

Goal: Print a triangle of numbers till n

E.g. For n = 4:

1

1 2

1 2 3

1 2 3 4

Build it with a nested loop.

Predict the output and find the mistakes

```
#include <stdio.h>

int main(void) {
            setvbuf(stdout, NULL, _IONBF, 0); // for buffer flushing, optional.

            int n=5;

            int i=0, j=0;

            while(i<= n) {

                        while(j <= i) {

                                    printf("%d ", j+1);

                                    j= j+1;
                        }

                        printf("\n");

                        i=i+1;
            }

            return 0;
}
```

Important

- Remember to check the boundary cases carefully in loops:
  - Boundary cases: the values of variables and behaviour of code at the first and last/terminating iteration of the loop

Corrected code(one version)

```
int n=5;
int i=1, j=1;

while(i<= n) {
        j=1;  //reset inner loop counter to the initial value
        while(j <= i) {
                printf("%d ", j);
                j= j+1;
        }

        printf("\n"); //note that this leaves an extra blank line
                        // Can you remove the extra blank line?
        i=i+1;
}
```

Recap: Nested Loops

- One loop within the other

- Can think of row variable, and column variable
  - Outer loop = row variable
  - Inner loop = column variable
  - Fix row. Execute inner loop for all iterations for a fixed row
  - Repeat for every row.

  - **For problems with prutor account creation, email: Mr. Shivakumar Marepally :** shivareddy.m@cse.iith.ac.in

Example 3:

- You retrieve a cuboidal sample of a rock of length $a \times b \times c$ cm^3. (assume a,b,c are integers)
- You have a probe that can check for contamination in a unit $(1\ \mathrm{cm}^3)$ volume and get a value $\in (0,1)$ for the contamination
- Goal: Find the fraction of contamination in the rock, taking contamination values as input.

Arrays

From one value to a collection

- A running sum or maximum can be found without storing every input.
- But some tasks need the values again later: printing them, reversing them, or comparing pairs.
- We need one name that can keep several related values.

Arrays

An array stores several values of the same type under one name.

Each stored value has a numbered position called an index.

Declaring an array

```
int marks[5];

int marks[5] = {78, 85, 92, 64, 88};

marks   -> array name
int     -> type of every element
5       -> capacity: five elements

All array elements have the same declared type.
```

Array indices

```
scores:   [ 78 ] [ 85 ] [ 92 ] [ 64 ] [ 88 ]
index :      0      1      2      3      4
```

scores[0] is the first element.

scores[4] is the last element.

For an array of capacity n, the last valid index is n - 1.

Valid positions in an array

```
int marks[5];
```

```
marks[0]     valid
marks[1]     valid
marks[2]     valid
marks[3]     valid
marks[4]     valid
marks[5]     not valid
```

C **does not reliably stop an out-of-bounds access**: Avoid accessing invalid elements.

Data entry into an array

```
int values[50];
int n;
int i = 0;

scanf("%d", &n);

while (i < n)
{
    scanf("%d", &values[i]);
    i = i + 1;
}
```

Precondition: 0 <= n and n <= 50.

Processing an array

Start with i = 0.

Use values[i] in the current iteration.

Update a sum, count, maximum, or other summary.

Increase i, and stop when i reaches n.

The condition i < n uses exactly the valid indices 0 through n - 1.

Array summaries

Question:

Declare an integer array with capacity 10.

Read elements into it as input.

Print the mean and maximum elements.

Find the output

```
int values[4] = {3, 8, 2, 7};
int i = 0;
int sum = 0;

while (i < 4)
{
    sum = sum + values[i];
    i = i + 2;
}

printf("%d\n", sum);
```

Q: What is printed?

Answer and trace

```
i = 0  ->  add values[0] = 3
i = 2  ->  add values[2] = 2
i = 4  ->  condition is false
```

The output is: 5

Only indices 0 and 2 are visited.

Common array errors

```
int values[10];
int i = 0;

while (i <= n) {
    scanf("%d", &values[i]);
    i = i + 1;
}
```

Q: Find the error in the above program

Summary and looking ahead

Nested loops: the inner loop completes work for each outer iteration.

Reset the inner-loop counter before every new outer iteration.

Arrays store related values at indices 0 through n - 1.

A while loop can read, print, and process an array.

Next week: for loops and array algorithms using the same ideas